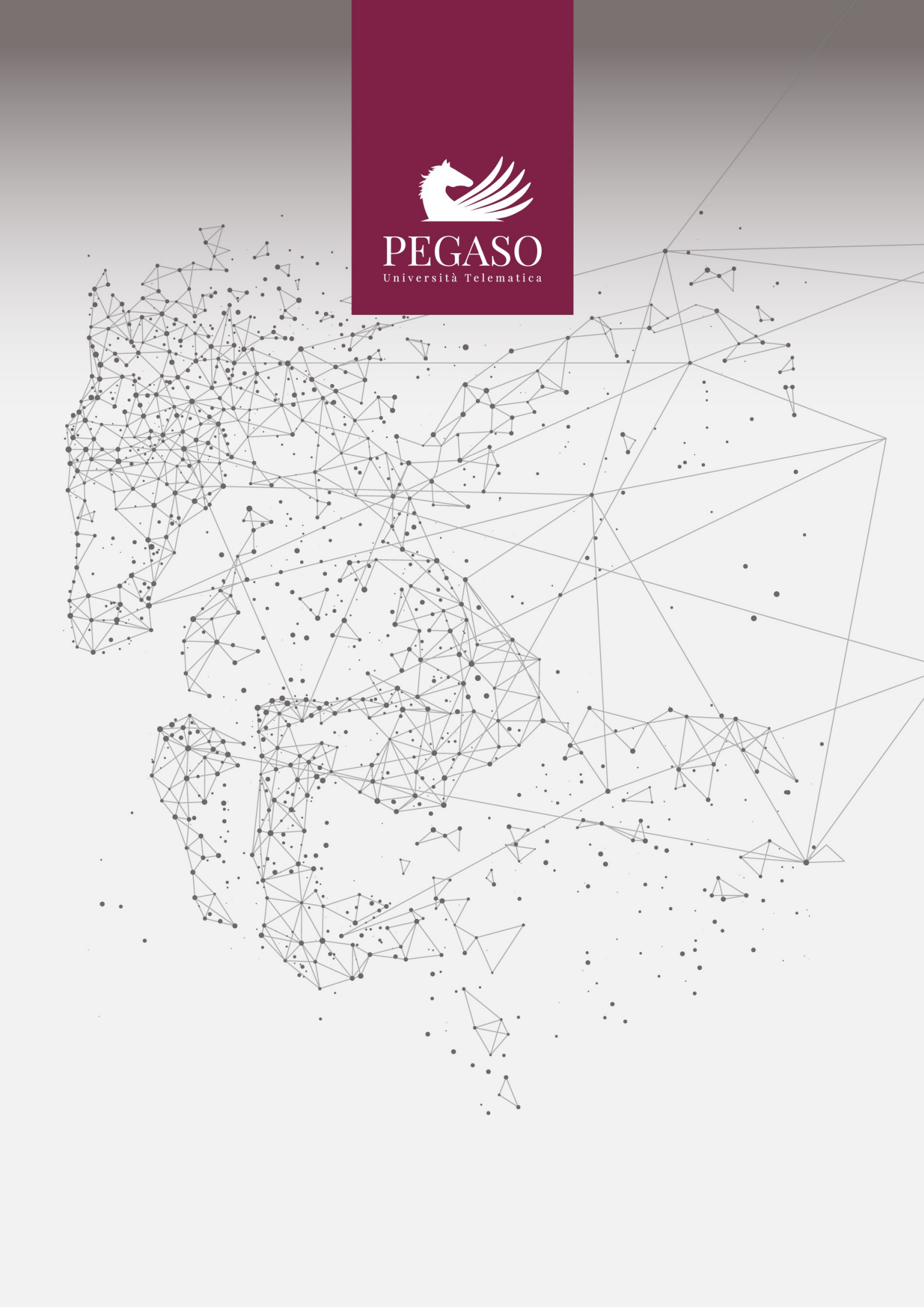

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. PIPELINING 3

2. CRITICITÀ NEL PIPELINING 6

3. MULTICORE E GPU 8

BIBLIOGRAFIA 10

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Pipelining

L'implementazione a ciclo singolo della CPU semplifica la presentazione della organizzazione strutturale e dell'interazione tra l'*Unità Centrale di Elaborazione* e l'*Unità di Controllo* che ne fanno parte, ma non è soddisfacente dal punto di vista delle prestazioni.

L'inefficienza deriva dal fatto che la durata del *ciclo di clock* deve essere tale da consentire di eseguire al suo interno tutte le operazioni relative alla esecuzione dell'istruzione di durata maggiore, determinando così uno spreco di tempo di inattività durante l'esecuzione di una istruzione che richiede una durata di tempo minore.

Il **PIPELINING** è una tecnica di implementazione Hardware della CPU basata su una metodologia di sovrapposizione temporale della esecuzione di più istruzioni le cui operazioni vengono svolte in uno stesso ciclo di *clock*.

L'organizzazione della esecuzione basata sul principio del *pipelining* è simile a quella di un lavoro per la realizzazione di un prodotto svolto attraverso una *catena di montaggio* che coinvolge reparti separati: ogni reparto riceve un prodotto parziale, esegue specifiche operazioni, quindi passa il prodotto ottenuto al reparto successivo, e così via fino al completamento di tutte le operazioni necessarie per realizzare il prodotto completo.

La strutturazione in una sequenza di fasi, detta ***pipeline***, del lavoro necessario per completare il prodotto e la separazione delle attrezzature utilizzate dai vari reparti, consentono di realizzare il PARALLELISMO caratteristico del *pipelining* in cui tutti i reparti lavorano contemporaneamente alla realizzazione di più prodotti. Il lavoro in *pipeline* è possibile se l'esecuzione del lavoro totale può essere organizzata in base a due requisiti fondamentali:

1. suddivisione in fasi da svolgere in sequenza
2. utilizzo di dispositivi fisici non condivisi.

Questo consente di incanalare i risultati parziali ottenuti dal lavoro svolto nelle varie fasi in un flusso continuo al cui termine si ottiene il risultato completo.

Ad esempio se si considera l'esecuzione di un bucato, si possono individuare quattro fasi che richiedono l'impiego di attrezzature diverse:

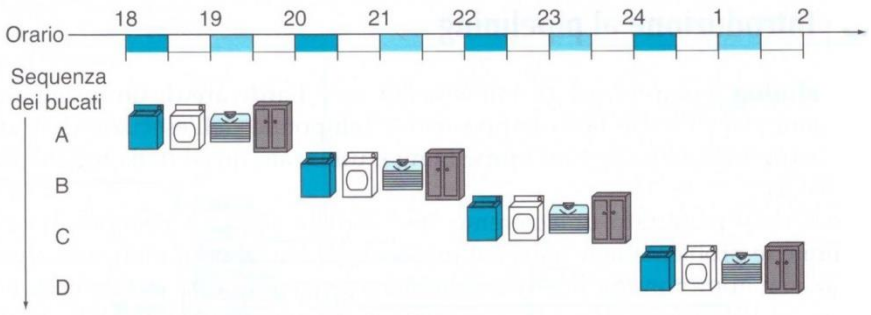
- lavaggio;
- asciugatura;
- stiratura;

PEGASO
Università Telematica

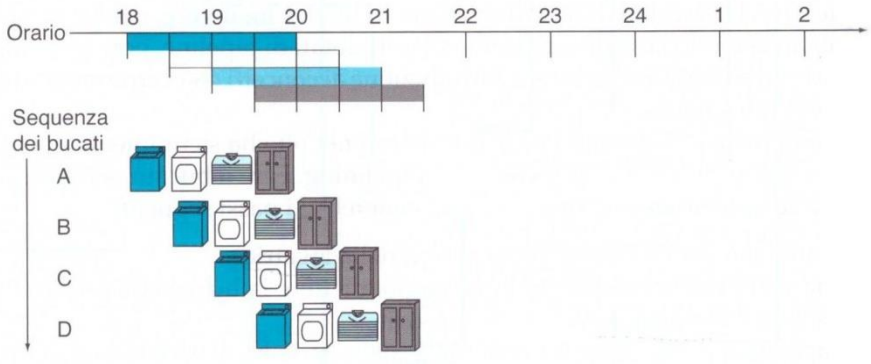
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- sistemazione nell'armadio.

Una esecuzione di più bucati può essere fatta eseguendo un bucato alla volta come mostrato nel seguente schema dove le varie fasi sono rappresentate da simboli iconici:



Poiché le operazioni relative al bucato successivo possono cominciare solo quando tutte le operazioni richieste dal bucato precedente sono state completate, si ha che ogni reparto deve aspettare inattivo il completamento delle altre fasi prima di poter eseguire la sua parte di lavoro relativa ad un successivo bucato. Una esecuzione in *pipeline* consiste nella sovrapposizione della esecuzione di più bucati come mostrato nel seguente schema:



Lo schema mostra il vantaggio dell'esecuzione parallela dei bucati offerto dalla tecnica di *pipelining*, sia per la riduzione del tempo di esecuzione di tutti i bucati, sia per il pieno utilizzo delle attrezzature.

Per applicare il principio del *pipelining* al caso DELL'ESECUZIONE DI UNA ISTRUZIONE, occorre innanzitutto individuare delle fasi che soddisfino i requisiti 1) e 2).

Il requisito 1) richiede la suddivisione dell'intero lavoro di esecuzione di una istruzione in fasi da svolgere in sequenza.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il requisito 2) richiede che ogni fase svolga operazioni che utilizzano dispositivi Hardware non utilizzati dalle altre fasi.

Nell'**ESECUZIONE di una ISTRUZIONE MIPS** evidenziata dalla implementazione a ciclo singolo della CPU, si possono individuare le seguenti **5 FASI** svolte in sequenza:

1. **Fetch** dell'istruzione, che consiste nel prelievo dell'istruzione dalla Memoria,
2. **Lettura** anticipata dei **registri** e **decodifica** del *codice operativo* da parte dell'***Unità di Controllo***,
3. **Calcolo** di un'operazione o di un indirizzo fatto dall'**ALU**,
4. Lettura (load) o scrittura (store) di un dato in Memoria,
5. Scrittura nel blocco dei Registri di un dato proveniente dalla Memoria (load) o dall'ALU (istruzioni aritmetico-logiche).

Inoltre, si può facilmente verificare che non vi è conflitto per l'utilizzazione di risorse Hardware in quanto istruzioni e dati sono contenuti in Memorie separate.

Nell'implementazione della CPU utilizzando la tecnica del pipelining le operazioni di ogni fase si svolgono in un ciclo di clock.

L'esecuzione della prima istruzione inizia con la fase di Fetch di tale istruzione nel primo ciclo di clock. Al termine di questa fase, nel secondo ciclo di clock, mentre la prima istruzione passa alla seconda fase di Lettura dei registri e Decodifica del codice operativo, inizia l'esecuzione della seconda istruzione con la relativa fase di Fetch, e così via concatenando le fasi di pipelining precedentemente descritte.

L'implementazione del pipelining richiede l'inserimento di ulteriori registri dedicati alla memorizzazione delle informazioni necessarie alla esecuzione di ogni istruzione nelle varie fasi, perché altrimenti questi valori sarebbero cancellati dai valori relativi alla esecuzione della successiva istruzione entrata in esecuzione nella pipeline.

Ad esempio, nell'esecuzione di una istruzione passando dalla fase di Fetch alla successiva occorre memorizzare sia i 32 bit della istruzione per disporre delle informazioni necessarie alla lettura dei registri ed alla decodifica del codice operativo, sia i 32 bit che rappresentano il valore nel Program Counter relativo a tale istruzione ancora da completare con l'esecuzione delle fasi successive.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Criticità nel pipelining

La esecuzione in parallelo delle istruzioni basata sullo schema delle fasi caratteristico del *pipelining* determina la possibilità che si presentino delle situazioni in cui le operazioni in una o più fasi della *pipeline* non possono essere effettuate.

Queste situazioni sono dette **criticità** (in inglese *hazard*) e determinalo il **blocco**, detto anche *stallo*, delle operazioni svolte in una fase fino a quando la causa che determina la situazione critica non viene eliminata.

Queste criticità determinano quindi un peggioramento delle prestazioni che si cerca di migliorare con l'implementazione del *pipelining*.

Le criticità possono essere di tre tipi.

CRITICITÀ STRUTTURALE

Si verifica quando le risorse Hardware presenti non sono in grado di supportare le operazioni richieste da diverse *fasi* della *pipeline* in esecuzione.

Ad esempio, se l'Hardware è strutturato con una sola Memoria che contiene sia le istruzioni che i dati, potrebbe determinarsi un conflitto per l'accesso in Memoria tra la fase di *Fetch* per il prelievo di una nuova istruzione e la fase di *Lettura/Scrittura* di dati in *Memoria* relativa alla esecuzione in parallelo di una istruzione nella *pipeline*.

Per evitare questo tipo di criticità la CPU del MIPS prevede due blocchi di Memoria separati per le istruzioni e i dati.

Nella progettazione delle istruzioni del *Linguaggio Macchina* si può tenere conto di questa criticità per favorire la implementazione del *pipelining*.

CRITICITÀ SUI DATI

Si verifica quando la *pipeline* deve essere messa in stallo perché una fase deve aspettare i risultati delle operazioni eseguite in un'altra fase della *pipeline*.

Ad esempio se nella *pipeline* sono entrate in esecuzioni le seguenti istruzioni:

```
add   $s0,  $t0,  $t1
sub   $s1,  $s0,  $t2
```

la scrittura in $s0 del risultato dell'addizione viene eseguita in una fase successiva alla fase 3 in cui l'ALU esegue il relativo calcolo.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Quindi la fase 2 della sottrazione successiva (contemporanea alla fase 3 della istruzione precedente) entra in stallo, perché per effettuare la lettura dell'operando in $s0 deve aspettare che vi venga scritto il valore dall'istruzione precedente.

Per evitare questo tipo di criticità si utilizza la tecnica detta di *propagazione* (*bypassing*) che consente all'istruzione successiva di leggere direttamente i risultati output calcolati dall'ALU senza dover attendere che siano memorizzati nel blocco dei Registri.

CRITICITÀ SUL CONTROLLO

Questa tipo di situazione è detta anche *criticità sui salti condizionati* poiché si verifica quando il prelievo di una istruzione dipende dall'esito del test di una istruzione di salto condizionato in esecuzione nella *pipeline*. Lo stallo della fase di *Fetch* della *pipeline* dovrebbe durare fino a quando l'esito del test non risulta disponibile e nel *Program Counter* viene scritto l'indirizzo dell'istruzione da prelevare, che può essere quello della istruzione successiva a quella di salto condizionato se il salto non avviene, o quello dell'istruzione a cui saltare.

La tecnica più semplice per ottimizzare le prestazioni nel caso di questo tipo di criticità consiste nel continuare l'esecuzione come se il salto non debba essere eseguito.

Questo consente di evitare senza danni lo stallo della *pipeline* se la condizione di salto risulta falsa e quindi effettivamente il salto non doveva essere effettuato.

Invece, nel caso contrario la *pipeline* deve essere corretta ristabilendo le condizioni per l'esecuzione del salto, il che significa che, allo stallo inevitabile dovuto all'attesa del risultato del test del salto condizionato, si aggiunge il tempo per ripristinare le condizioni iniziali.

Si sono anche sviluppate tecniche più sofisticate dette di ***predizione dei salti*** per gestire queste situazioni, che possono fornire una previsione corretta anche superiore al 90% dei casi.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Multicore e GPU

Il grande impulso dato al miglioramento delle prestazioni attraverso l'aumento della *frequenza di clock* ha trovato un limite fisico nella massima quantità di calore dissipabile dai sistemi di raffreddamento montati sui computer.

La tecnica del *pipelining* ha consentito di ottenere miglioramenti delle prestazioni introducendo la possibilità di eseguire in parallelo più istruzioni simultaneamente su una **singola CPU**.

Questo tipo di parallelismo introdotto nella progettazione Hardware è del tutto nascosto agli occhi del programmatore, che continua a progettare i programmi in un'ottica di esecuzione sequenziale delle istruzioni.

Una ulteriore metodologia di introduzione del parallelismo nella progettazione Hardware è la progettazione di Architetture che prevedano la presenza di **più di una CPU**.

Questo tipo di Architettura viene denominato con il termine **multi-core** (multi-processore) dove la parola ***core*** (nucleo) è adottata come sinonimo di *processore* (CPU).

Il termine *multi-core* è generico e può essere affiancato anche con altri termini specifici riferiti al numero di CPU presenti, quali *dual-core*, *quad-core*, ecc. (2 CPU, 4 CPU, ecc.).

I primi computer *dual-core* vennero commercializzati da Intel nel 2005.

A differenza del parallelismo introdotto dal *pipelining*, lo sfruttamento delle potenzialità di miglioramento delle prestazioni offerte dalle Architetture *multi-core* richiede che il programmatore tenga conto della diversa strutturazione dell'Hardware. Infatti le prestazioni sono migliorate se tutti i *core* sono pienamente impegnati nella esecuzioni di compiti che parallelamente concorrono alla esecuzione di un programma.

Quindi un programmatore deve suddividere il suo programma in modo tale che ciascun *core* abbia all'incirca lo stesso carico di lavoro nello stesso intervallo di tempo, e che il tempo aggiuntivo richiesto per la pianificazione delle attività e per il coordinamento dell'esecuzione dei diversi compiti non superi il risparmio ottenuto con l'esecuzione parallela.

Un campo in cui la possibilità di sfruttare il parallelismo offerto dalle Architetture *multi-core* ha portato notevoli miglioramenti delle prestazioni è costituito dalle ***Unità di elaborazione grafica*** denotate con l'acronimo **GPU** (*Graphic Processing Unit*).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Lo sviluppo sempre crescente delle applicazioni di grafica 2D e 3D e dei videogiochi ha portato alla realizzazione di GPU come processori specializzati in questo tipo di elaborazioni affiancati come *co-processori* alla CPU o implementati nel medesimo circuito integrato. Sono anche commercializzati processori *multi-core* dotati di più *core* di GPU.

Attualmente le GPU ad alto parallelismo hanno cominciato a contendere alla CPU funzioni computazionali offrendo la possibilità di programmare un insieme sempre più ampio di applicazioni.

Questo più ampio impiego delle GPU viene denominato

General Purpose GPU (GPU ad utilizzo generico) accompagnato anche dall'introduzione di linguaggi di programmazione ispirati al Linguaggio C che consentono di scrivere programmi eseguibili direttamente da una GPU.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- D. A. Patterson, J. L. Hennessy, "Struttura e progetto dei calcolatori", Zanichelli.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).